

Rapport : Implémentation d'un Algorithme Apriori Hybride

1. Introduction

L'algorithme Apriori est l'un des algorithmes les plus connus pour extraire des règles d'association dans les bases de données transactionnelles. Cependant, il présente plusieurs limites importantes en pratique.

Dans ce travail, nous avons implémenté une version Hybride de l'algorithme Apriori en combinant trois améliorations afin de le rendre plus efficace, plus flexible et mieux adapté aux données réelles.

2. Limites de l'Apriori Classique

L'algorithme Apriori classique souffre des problèmes suivants :

- Seuil de support fixe (minsup) : On utilise le même seuil pour tous les niveaux, ce qui fait qu'on rate soit beaucoup de motifs rares, soit on génère trop de motifs inutiles.
- Génération massive de candidats : À chaque niveau, le nombre de candidats explose, ce qui augmente fortement le temps d'exécution et la consommation mémoire.
- Ne prend pas en compte la rareté des items : Tous les items sont traités de la même façon, même si certains sont très rares.
- Difficulté à choisir le minsup : Un minsup mal choisi donne soit trop peu de résultats, soit trop de résultats bruyants.
- Pas adapté aux itemsets longs : Les itemsets de grande taille ont naturellement un support plus faible, mais l'algorithme classique les élimine souvent.

3. Présentation de notre Apriori Hybride

Nous avons développé une version améliorée appelée Apriori Hybride qui combine trois techniques :

a. Minsup adaptatif initial

Au lieu d'utiliser un seuil de support fixe et arbitraire comme dans l'Apriori classique, nous calculons automatiquement un **minsup initial** adapté aux données.

Méthode :

- Nous calculons d'abord le support de chaque item individuel (1-itemset).
- Nous calculons ensuite la **moyenne** de ces supports (avg_sup).
- Le minsup initial est fixé à **85 % de cette moyenne**, avec une valeur minimale de 0,15 (15 %) pour éviter un seuil trop bas.

Avantage : Ce seuil de départ est plus réaliste et mieux adapté à la distribution réelle des données, ce qui permet de démarrer l'algorithme avec un seuil ni trop strict, ni trop permissif.

b. Support Minimum par Item (MIS - Minimum Item Support)

Dans l'Apriori classique, tous les items doivent respecter le même seuil de support minimum. Cela pose problème car certains items sont très fréquents tandis que d'autres sont rares.

Notre solution : Nous utilisons un **Support Minimum par Item (MIS)**. Chaque item possède son propre seuil de support, calculé en fonction de sa fréquence réelle dans les données.

Méthode :

- Pour chaque item, on calcule son support individuel.
- On lui attribue un seuil égal à **55 % de son support** (base_factor = 0.55).
- Un plancher de **3 % (0.03)** est appliqué pour éviter des seuils trop bas.
- Avantage : Cette approche permet aux items rares d'apparaître dans des règles intéressantes, tout en restant exigeante pour les items très fréquents. Elle rend l'algorithme plus flexible et réaliste.

c. Diminution dégressive du minsup selon la taille k

Dans l'Apriori classique, le seuil de support minimum (minsup) reste fixe quel que soit le niveau k. Cela pose problème car les itemsets de grande taille ont naturellement un support plus faible que les itemsets de petite taille, ce qui conduit à leur élimination prématurée.

Notre solution : Nous appliquons une **diminution progressive du seuil de support** à mesure que la taille k des itemsets augmente.

Méthode :

Date : 09 Avril 2026

- À chaque niveau k ($k \geq 2$), le seuil minsup_k est recalculé selon la formule :

$$\text{minsup}_k = \max(\text{minsup_initial} - (k - 1) \times \text{attenuation}, \varepsilon)$$

où $\text{attenuation} = 0.04$ et $\varepsilon = 1e-6$ (valeur minimale pour éviter un seuil nul).

- Le seuil initial minsup_initial est calculé automatiquement (85 % du support moyen des items).
- Exemple issu de notre exécution :
 - $k=1 \rightarrow \text{minsup} = 0.2125$
 - $k=2 \rightarrow \text{minsup} = 0.1725$
 - $k=3 \rightarrow \text{minsup} = 0.1325$
 - $k=4 \rightarrow \text{minsup} = 0.0925$
 - $k=5 \rightarrow \text{minsup} = 0.0525$

Avantage : Cette approche permet de conserver des itemsets longs potentiellement intéressants malgré leur faible support, tout en évitant une explosion du nombre de candidats aux niveaux inférieurs. La décroissance linéaire et contrôlée (via le paramètre attenuation) offre un bon équilibre entre exhaustivité et performance.

4. Améliorations Apportées et Solutions

Problème de l'Apriori Classique	Solution dans l'Apriori Hybride	Bénéfice obtenu
Seuil fixe (minsup)	Minsup adaptatif + MIS par item	Meilleure adaptation aux données
Traitement égal des items rares et fréquents	Support Minimum Individuel (MIS)	Découverte de motifs rares utiles
Élimination trop rapide des itemsets longs	Diminution progressive du minsup avec k	Meilleure extraction de règles longues
Explosion du nombre de candidats	Meilleure taille initiale + diminution dégressive	Réduction du temps et de la mémoire
Choix manuel difficile du minsup	Calcul automatique du minsup initial	Moins de paramétrage manuel

5. Résultats Obtenus

Sur le jeu de données `user\_behavior\_dataset\_cleaned.csv` (701 transactions, 40 items) :

- Nombre d'itemsets fréquents trouvés : 142
- Nombre de règles d'association générées : 788
- Temps d'exécution : 0.0519 seconds
- Mémoire utilisée : "memory_used_mb": 0.07, "peak_memory_mb": 123.05,

Les règles les plus fortes montrent des associations très cohérentes (exemple : Beaucoup d'apps
→ Comportement intensif avec confiance = 1.0 et lift élevé).

6. Conclusion

L'Apriori Hybride que nous avons développé surpasse clairement la version classique en termes de flexibilité, qualité des résultats et adaptation aux données réelles. Les trois améliorations (minsup adaptatif, MIS, et diminution dégressive) permettent de résoudre les principaux défauts de l'algorithme original tout en gardant sa logique de base.

Cette approche est particulièrement utile pour l'analyse du comportement utilisateur comme dans ce projet.